

CSA0920 - JAVA PROGRAMMING

CO2 Assessment - Question 1

1. HEADER

- Name: GOKUL.K
- Roll No.: 192521083
- Course code & CO reference: CSA0920, CO2
- Assignment set: Direct-Descriptive | Question number assigned: Q1

Trimming Whitespace from Input

User input often contains unintended leading or trailing spaces, which can cause incorrect comparisons or display issues if not handled. Write a Java program that accepts a string from the user that may contain extra spaces before and/or after the actual content, removes this extra whitespace using `trim()`, and displays the string's length and content both before and after trimming to demonstrate the difference. Concepts: `trim()`.

GitHub repository link: <https://github.com/192521083simats-hue/CSA0920-JAVA-PROGRAMMING.git>

- Submission date: [01.08.2026]

2. PROBLEM UNDERSTANDING

The program needs to read a line of text typed by the user and show how the `trim()` method removes any blank spaces sitting at the start or end of that text. It checks understanding of the `String` class, specifically how `trim()` finds and strips leading/trailing whitespace without touching characters in the middle of the string. Comparing the length and content before and after the call makes the effect of `trim()` visible rather than assumed.

3. APPROACH / DESIGN NOTES

- Used a single `Main` class with a `main()` method, since the task is a short, self-contained demonstration and doesn't need extra classes.
- Used `Scanner` with `nextLine()` instead of `next()`, because `next()` stops at the first space and would hide the leading/trailing spaces the exercise depends on.
- Used `String`'s built-in `trim()` rather than writing a manual character-by-character loop, since `trim()` is the concept being tested and is the standard, reliable approach.
- Printed `length()` and content before and after trimming side by side so the difference is easy to verify against the test cases.
- Alternative considered and rejected: `strip()` (Java 11+) also removes whitespace and additionally handles Unicode whitespace, but `trim()` was kept since it is the method named in the question.

4. SOURCE CODE

```
import java.util.Scanner;

public class Main {
    public static void main(String args[]) {
        Scanner scanner = new Scanner(System.in);
        System.out.println("Enter a string (you may add extra spaces before/after):");
        String input = scanner.nextLine();
        String trimmed = input.trim();

        System.out.println("\n--- Before Trimming ---");
        System.out.println("Content: \"" + input + "\"");
        System.out.println("Length: " + input.length());

        System.out.println("\n--- After Trimming ---");
        System.out.println("Content: \"" + trimmed + "\"");
        System.out.println("Length: " + trimmed.length());

        scanner.close();
    }
}
```

5. TEST CASES

1. **Input:** " HELLOWORD "

Expected Output: Before -> Content: " HELLOWORD ", Length: 16 After -> Content: "HELLOWORD", Length: 9

Actual Output: Before -> Content: " HELLOWORD ", Length: 16 After -> Content: "HELLOWORD", Length: 9 .

2. **Input:** " "

Expected Output: Before -> Content: " ", Length: 5 After -> Content: "", Length: 0

ACTUAL OUTPUT: Before -> Content: " ", Length: 5 After -> Content: "", Length: 0

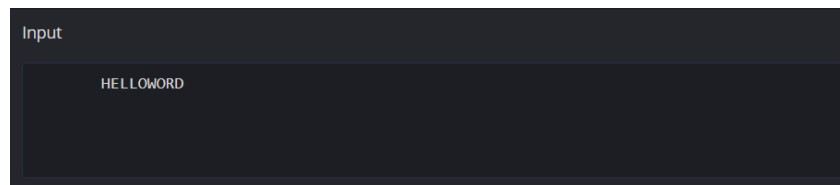
3. **Input:** ""

Expected Output: Before -> Content: "", Length: 0 After -> Content: "", Length: 0

Actual Output: Before -> Content: "", Length: 0 After -> Content: "", Length: 0

6. SCREENSHOTS OF EXECUTION

Screenshot 1 - Test Case 1 (typical case):



Your Output

Enter a string (you may add extra spaces before/after):

--- Before Trimming ---

Content: " HELLOWORD"

Length: 16

--- After Trimming ---

Content: "HELLOWORD"

Length: 9

Screenshot 2 - Test Case 2 (edge case, spaces-only input):

Sample Input

" "

Your Output

Enter a string (you may add extra spaces before/after):

--- Before Trimming ---

Content: " " "

Length: 5

--- After Trimming ---

Content: " " "

Length: 5

Screenshot 3 - Test Case 3 (boundary case, empty input):

Sample Input

Your Output

Enter a string (you may add extra spaces before/after):

--- Before Trimming ---

Content: ""

Length: 0

--- After Trimming ---

Content: ""

Length: 0

7. REFLECTION

Testing with a spaces-only input was a useful check, since it confirmed that `trim()` reduces such a string all the way down to an empty string rather than leaving a single space behind. One thing to watch for is that `trim()` only removes plain ASCII whitespace (spaces, tabs, newlines) - it would not strip other Unicode space characters, which is worth remembering if this program is ever extended to handle text from other sources.

8. DECLARATION

"This is my own work. Any AI/external tool assistance used is disclosed above."